



计算机系统原理实验报告

第四次实验

基于 RISC-V 开发板的加密算法性能测试与加速比分析

成员：	江李阳	202400460042
	王健一	202400460037
	邱泽辉	202400460046
	史淇	202333180074
专业：	密码科学与技术	
日期：	June 10, 2026	

Contents

1	实验目的	2
2	实验环境	2
3	实验原理	2
3.1	SM4 算法	2
3.1.1	加密算法	2
3.1.2	轮密钥生成方案	3
3.1.3	解密算法	3
4	实验过程	4
4.1	实验环境搭建	4
4.2	SM4 示例程序的测试使用	5
4.3	SM3 示例程序的使用测试	6
4.4	SM4 默认循环与循环展开	6
4.4.1	SM4 默认循环	6
4.4.2	SM4 循环展开	7
4.5	SM4 默认 XBox 查表与 XBox 合并	8
4.5.1	SM4 默认 XBox 查表	9
4.5.2	XBox 合并	10
4.6	SM4 默认轮函数连续异或与减少指令依赖	12
4.7	默认字节序转换与 RISC-V B 拓展 rev8 转换	13

1 实验目的

1. 学习并掌握 RISC-V 加架构的开发板 (香橙派) 的基本使用方法。
2. 学习 OpenHiTLS 加密算法的使用示例, 能正确运行 SM3 或 SM4 实例。
3. 记录各方案的运行时间, 吞吐率以及加速比。
4. 结合技术方案差异、开发板硬件条件、系统环境和测试方法等, 对实验结果进行分析。
5. 若某一技术方案依赖开发板不支持的指令扩展, 应说明原因并选择替代方案。

2 实验环境

- 主机硬件环境
 - 处理器:13th Gen Intel(R) Core(TM) i5-13420H
 - 内存:16 GB
- 主机软件环境
 - CP210x Windows Drivers(USB 转 TTL 串口芯片驱动)
 - MobaXterm(终端管理工具)
 - OpenSSH_for_Windows_9.5p2, LibreSSL 3.8.2(远程访问开发板)

3 实验原理

3.1 SM4 算法

SM4 算法采用非平衡 Feistel 结构, 分组长度为 128bit, 密钥长度为 128bit, 迭代轮数为 32 轮。

3.1.1 加密算法

输入 128bit 明文 m , 按 32-bit 分为 4 个字 $X_0^0, X_0^1, X_0^2, X_0^3$ 进行 32 轮迭代, 每轮迭代都有一个 32bit 的轮密钥 K_{i-1} 参与计算, 第 $i(i = 1, 2, \dots, 32)$ 轮的计算如下:

$$\begin{aligned}X_i^0 &= X_{i-1}^1 \\X_i^1 &= X_{i-1}^2 \\X_i^2 &= X_{i-1}^3 \\X_i^3 &= X_{i-1}^0 \oplus T(X_{i-1}^1 \oplus X_{i-1}^2 \oplus X_{i-1}^3 \oplus K_{i-1})\end{aligned}$$

对最后一轮迭代的输出进行反序变化 R 得到 128bit 密文 c :

$$c = (Y^0, Y^1, Y^2, Y^3) = R(X_{32}^0, X_{32}^1, X_{32}^2, X_{32}^3) = (X_{32}^3, X_{32}^2, X_{32}^1, X_{32}^0)$$

其中函数 $T: \mathbb{F}_2^{32} \rightarrow \mathbb{F}_2^{32}$ 由非线性变换 S 和线性变换 L 组成:

$$T = L \circ S$$

设函数 T 的 32bit 输入为 $A = (a_0, a_1, a_2, a_3) \in (\mathbb{F}_{2^8})^4$, 则

- 非线性变换 S : 对每个字节单独进行查表代替操作, 即

$$B = (S(a_0), S(a_1), S(a_2), S(a_3))$$

类似 AES 加密, SM4 算法采用 1 个 S 盒, 查表时将每个字节的高 4bit 看作行标, 低 4bit 看作列标, 对应取值即为输出字节。

- 线性变换 L : 对非线性变换 S 的 32bit 输出 B , 计算

$$L(B) = B \oplus (B \lll 2) \oplus (B \lll 10) \oplus (B \lll 18) \oplus (B \lll 24)$$

其中 $\lll x$ 表示循环左移 x bit。

3.1.2 轮密钥生成方案

SM4 算法的主密钥 K 为 128bit, 轮密钥生成方案也采用非平衡 Feistel 结构。轮密钥 K_i 由主密钥 K 通过密钥扩展算法生成, 具体做法为: 先将主密钥 K 按 32-bit 分为 4 个字 MK_0, MK_1, MK_2, MK_3 , 然后与 32-bit 常量 FK_i 进行异或得到初始轮密钥:

$$K_{-4} = MK_0 \oplus FK_0$$

$$K_{-3} = MK_1 \oplus FK_1$$

$$K_{-2} = MK_2 \oplus FK_2$$

$$K_{-1} = MK_3 \oplus FK_3$$

其中 $FK_0 = 0xA3B1BAC6, FK_1 = 0x56AA3350, FK_2 = 0x677D9197, FK_3 = 0xB27022DC$ 。接着通过迭代计算得到后续轮密钥:

$$K_i = K_{i-4} \oplus T'(K_{i-3} \oplus K_{i-2} \oplus K_{i-1} \oplus CK_i), \quad i = 0, 1, \dots, 31$$

其中 CK_i 为固定参数; T' 函数与 T 函数类似, 但线性变换部分不同, 仅需替换 L 为 L' :

$$T'(X) = L'(S(X))$$

$$L'(B) = B \oplus (B \lll 13) \oplus (B \lll 23)$$

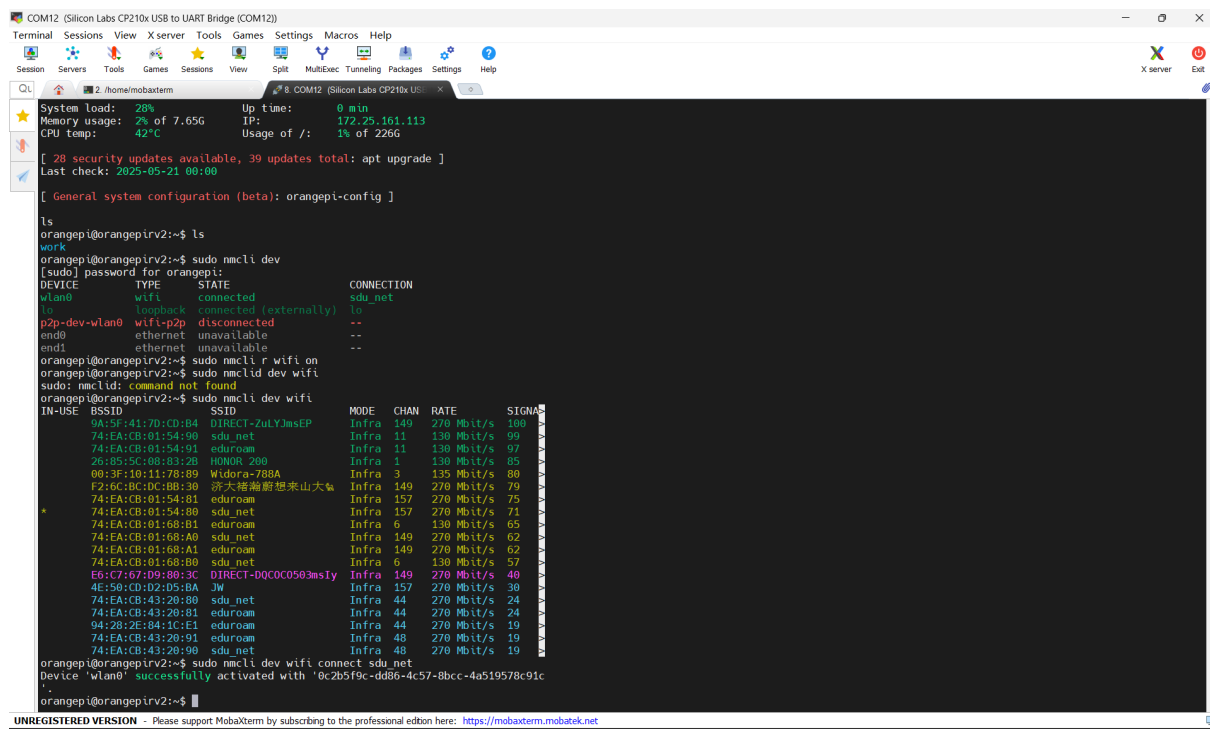
3.1.3 解密算法

因为采用广义 Feistel 结构, 所以 SM4 算法具有加解密结构一致性, 因此解密算法与加密算法一致, 仅需将参与第 i ($i = 1, 2, \dots, 32$) 轮迭代运算的轮密钥由 K_{i-1} 替换为 K_{32-i} 即可。

4 实验过程

4.1 实验环境搭建

为香橙派开发板插上 TYPEC 电源适配器并用 USB 转 TTL 模块连接开发板与主机, 连接完成后使用 MobaXterm 访问开发板, 终端连接 WIFI 成功如图:



```
COM12 (Silicon Labs CP210x USB to UART Bridge (COM12))
Terminal Sessions View Xserver Tools Games Settings Macros Help
Session Servers Tools Games Sessions View Split MultiExec Tunneling Packages Settings Help
System load: 20% Up time: 0 min
Memory usage: 2% of 7.65G IP: 172.25.161.113
CPU temp: 42°C Usage of /: 1% of 226G
[ 28 security updates available, 39 updates total: apt upgrade ]
Last check: 2025-05-21 00:00
[ General system configuration (beta): orangepi-config ]

ls
orangepi@orangepirv2:~$ ls
work
orangepi@orangepirv2:~$ sudo nmcli dev
[sudo] password for orangepi:
DEVICE TYPE STATE CONNECTION
wlan0 wifi connected sdu_net
lo loopback connected (externally) lo
d2p-dev-wlan0 wifi-d2p disconnected (externally) --
end0 ethernet unavailable --
end1 ethernet unavailable --

orangepi@orangepirv2:~$ sudo nmcli r wifi on
orangepi@orangepirv2:~$ sudo nmcli dev wifi
sudo: nmcli: command not found
orangepi@orangepirv2:~$ sudo nmcli dev wifi
IN-USE BSSID SSID MODE CHAN RATE SIGNAL
9A:5F:41:7D:CD:B4 DIRECT-ZuLYJmsEP Infra 149 270 Mbit/s 100
74:EA:CB:01:54:90 sdu_net Infra 11 130 Mbit/s 99
74:EA:CB:01:54:91 eduroam Infra 11 130 Mbit/s 97
26:05:5C:00:83:20 HONOR 200 Infra 1 130 Mbit/s 85
00:3F:10:11:78:89 Widora-788A Infra 3 135 Mbit/s 80
F2:6C:8C:DC:B8:30 济大格瀚游想来山大 WiFi Infra 149 270 Mbit/s 79
74:EA:CB:01:54:01 eduroam Infra 157 270 Mbit/s 75
74:EA:CB:01:54:80 sdu_net Infra 157 270 Mbit/s 71
74:EA:CB:01:68:B1 eduroam Infra 6 130 Mbit/s 65
74:EA:CB:01:68:A0 sdu_net Infra 149 270 Mbit/s 62
74:EA:CB:01:68:A1 eduroam Infra 149 270 Mbit/s 62
74:EA:CB:01:68:B0 sdu_net Infra 6 130 Mbit/s 57
E6:C7:67:D9:80:3C DIRECT-D0C0C0503msTy Infra 149 270 Mbit/s 40
4E:5B:CB:D2:D5:BA JW Infra 157 270 Mbit/s 30
74:EA:CB:43:20:80 sdu_net Infra 44 270 Mbit/s 24
74:EA:CB:43:20:81 eduroam Infra 44 270 Mbit/s 24
94:2B:2E:94:1C:E1 eduroam Infra 44 270 Mbit/s 19
74:EA:CB:43:20:91 eduroam Infra 48 270 Mbit/s 19
74:EA:CB:43:20:90 sdu_net Infra 48 270 Mbit/s 19

orangepi@orangepirv2:~$ sudo nmcli dev wifi connect sdu_net
Device wlan0 successfully activated with '0c2b5f9c-dd86-4c57-8bcc-4a519578c91c'
orangepi@orangepirv2:~$
```

Figure 1: 开发板连接 WIFI

考虑到原实验为串口连接开发板进行单人次实验, 而实际实验小组有多人, 轮流使用串口会显著增加实验总耗时, 不方便调试, 也不便于课后小组成员随时随地使用开发板的需要, 因此我们为开发板配置网络后启动了 OpenSSH 服务。基本实现为借助 Tailscale 工具将包括开发板在内的所有主机添加到同一个虚拟局域网中, 使得所有设备都可以跨局域网登录访问开发板:

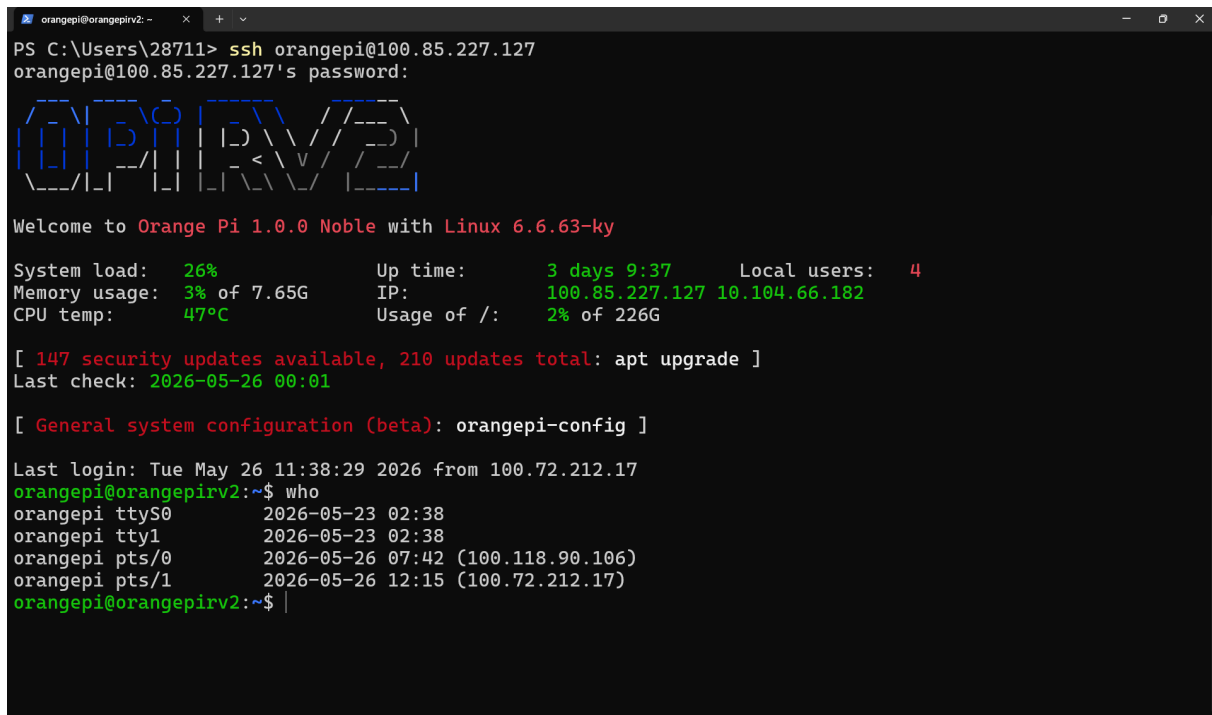


Figure 2: 远程连接开发板

这里远程连接只是为了方便多用户远程访问开发板和编写代码, 为确保实验结果准确性, 每次测试算法效率时仅一位用户在使用。

4.2 SM4 示例程序的测试使用

按照实验指导书的指示分别安装依赖、获取 libboundscheck 源码并编译、获取 openhtls 源码并编译、然后运行实例代码,结果如下图:

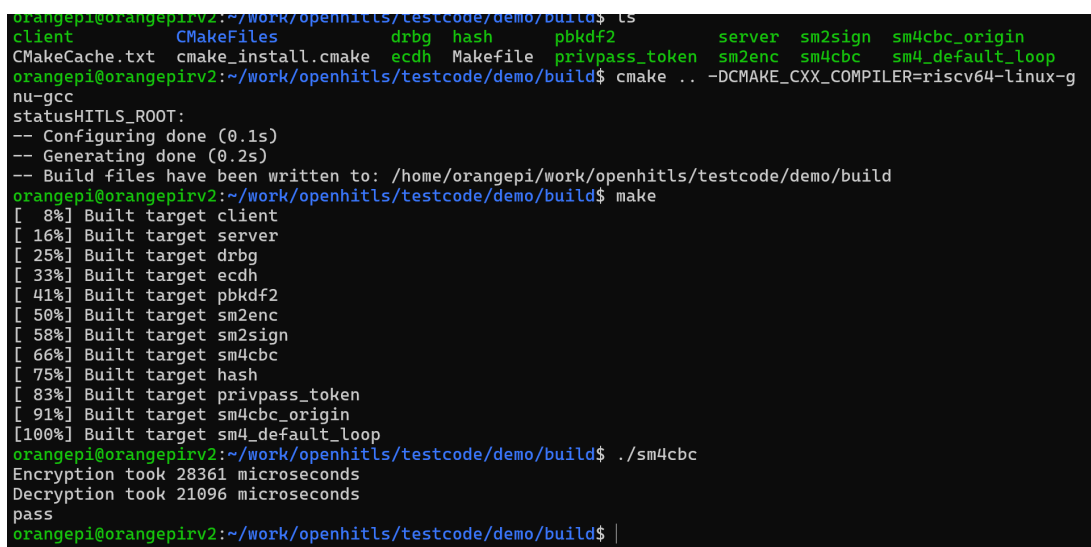


Figure 3: SM4 示例程序测试运行

4.3 SM3 示例程序的使用测试

修改 hash.c 文件中的部分代码, 重新编译即可测试运行 SM3 示例程序, 结果如下:

Figure 4: 修改代码

Figure 5: sm3 示例程序测试运行

4.4 SM4 默认循环与循环展开

4.4.1 SM4 默认循环

按照课本介绍 SM4 算法的实现, 不采用任何优化策略, 完整实现 SM4 的 32 轮迭代并在每一轮计算中对明文分组做非线性变换 S 和线性变换 L, 记录算法运行时间, 并以此组作为基准组, 时间开销主要在 32 轮迭代以及每轮迭代中的 S 和 L 操作:

```
1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X[4];
6     sm4LoadBlock(in, X);
7
8     for (int i = 0; i < 32; i++) {
9         uint32_t tmp = X[1] ^ X[2] ^ X[3] ^ rk[i];
10        uint32_t t = sm4T(tmp);
11        uint32_t newX0 = X[0] ^ t;
12        X[0] = X[1];
13        X[1] = X[2];
14        X[2] = X[3];
15        X[3] = newX0;
16    }
17
18    sm4StoreBlock(X, out);
19 }
```

结果如下:

其中测试输入数据大小为 1MB。

4.4.2 SM4 循环展开

上述的循环次数以及每次循环的操作是已知的, 使用宏一次性生成全部 32 轮代码, 可以去掉 32 次 `i++` 和条件判断, 避免分支预测失败惩罚, 同时再使用数组而是四个变量即可实现 32 轮循环迭代, 省去计算数组索引和基于偏移量取数的开销, 关键代码如下:

```
1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     uint32_t tmp, t, newX;
7
8     /* Manual load to scalars for unrolled access */
9     {
10         uint32_t arr[4];
11         sm4LoadBlock(in, arr);
12         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
13     }
14
15     #define SM4_UNROLLED_ROUND(N) do {           \
16         tmp = X1 ^ X2 ^ X3 ^ rk[N];             \
17         t    = sm4T(tmp);                        \
18         newX = X0 ^ t;                          \
19         X0 = X1; X1 = X2; X2 = X3; X3 = newX;    \
20     } while(0)
21
22     SM4_UNROLLED_ROUND(0);
23     SM4_UNROLLED_ROUND(1);
24     SM4_UNROLLED_ROUND(2);
25     SM4_UNROLLED_ROUND(3);
26     SM4_UNROLLED_ROUND(4);
27     SM4_UNROLLED_ROUND(5);
28     SM4_UNROLLED_ROUND(6);
29     SM4_UNROLLED_ROUND(7);
30     SM4_UNROLLED_ROUND(8);
31     SM4_UNROLLED_ROUND(9);
32     SM4_UNROLLED_ROUND(10);
33     SM4_UNROLLED_ROUND(11);
34     SM4_UNROLLED_ROUND(12);
35     SM4_UNROLLED_ROUND(13);
36     SM4_UNROLLED_ROUND(14);
37     SM4_UNROLLED_ROUND(15);
38     SM4_UNROLLED_ROUND(16);
39     SM4_UNROLLED_ROUND(17);
40     SM4_UNROLLED_ROUND(18);
41     SM4_UNROLLED_ROUND(19);
```

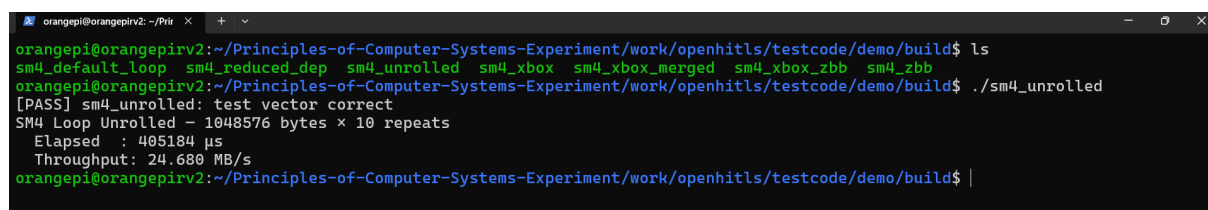


```

42     SM4_UNROLLED_ROUND(20);
43     SM4_UNROLLED_ROUND(21);
44     SM4_UNROLLED_ROUND(22);
45     SM4_UNROLLED_ROUND(23);
46     SM4_UNROLLED_ROUND(24);
47     SM4_UNROLLED_ROUND(25);
48     SM4_UNROLLED_ROUND(26);
49     SM4_UNROLLED_ROUND(27);
50     SM4_UNROLLED_ROUND(28);
51     SM4_UNROLLED_ROUND(29);
52     SM4_UNROLLED_ROUND(30);
53     SM4_UNROLLED_ROUND(31);
54
55     #undef SM4_UNROLLED_ROUND
56
57     /* Reverse transform & store */
58     {
59         uint32_t arr[4] = {X0, X1, X2, X3};
60         sm4StoreBlock(arr, out);
61     }
62 }

```

这里所有宏会在编译阶段就计算出来, 能降低运行时计算的时间开销。运行结果如下:



```

orangeipi@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop  sm4_reduced_dep  sm4_unrolled  sm4_xbox  sm4_xbox_merged  sm4_xbox_zbb  sm4_zbb
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_unrolled
[PASS] sm4_unrolled: test vector correct
SM4 Loop Unrolled - 1048576 bytes x 10 repeats
Elapsed   : 405184 µs
Throughput: 24.680 MB/s
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$

```

Figure 6: SM4 循环展开

可以看到循环展开能有效提升 SM4 加密算法的运行效率, 提升吞吐量。

4.5 SM4 默认 XBox 查表与 XBox 合并

在 SM4 加密算法的每轮循环中需要对输入的每个字做非线性的过 S 盒操作, 和线性的 L 操作, 对于

$$X_3 = a_0 || a_1 || a_2 || a_3$$

为最后一个 32bit 分组, 分别过 S 盒后得到

$$B = b_0 || b_1 || b_2 || b_3 = Sbox(a_0) || Sbox(a_1) || Sbox(a_2) || Sbox(a_3)$$

在代码中通过移位拼接:

$$B = (b_0 \ll 24) \oplus (b_1 \ll 16) \oplus (b_2 \ll 8) \oplus (b_3)$$

由于 L 是线性操作, 满足

$$L(a \oplus b) = L(a) \oplus L(b)$$

因此

$$\begin{aligned}
 L(B) &= L(b_0 \ll 24) \oplus L(b_1 \ll 16) \oplus L(b_2 \ll 8) \oplus L(b_3) \\
 &= B \oplus \text{ROL}(B, 2) \oplus \text{ROL}(B, 10) \oplus \text{ROL}(B, 18) \oplus \text{ROL}(B, 24) \\
 &= \text{ROL}(Xbox[b_0], 24) \oplus \text{ROL}(Xbox[b_1], 16) \oplus \text{ROL}(Xbox[b_2], 8) \oplus Xbox[b_3]
 \end{aligned} \tag{1}$$

4.5.1 SM4 默认 Xbox 查表

因此预计算 $Xbox[i] = L(S(i))$ 共 $2^8 = 256$ 项, 最后循环移位并异或即可得到最终结果。关键代码如下:

```

1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     {
7         uint32_t arr[4];
8         sm4LoadBlock(in, arr);
9         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
10    }
11
12    initXbox();
13
14    for (int i = 0; i < 32; i++) {
15        uint32_t t = X1 ^ X2 ^ X3 ^ rk[i];
16        uint8_t b0 = (uint8_t)((t >> 24) & 0xFF);
17        uint8_t b1 = (uint8_t)((t >> 16) & 0xFF);
18        uint8_t b2 = (uint8_t)((t >> 8) & 0xFF);
19        uint8_t b3 = (uint8_t)(t & 0xFF);
20
21        uint32_t newX0 = X0 ^ ROL(xbox[b0], 24)
22                        ^ ROL(xbox[b1], 16)
23                        ^ ROL(xbox[b2], 8)
24                        ^ xbox[b3];
25
26        X0 = X1; X1 = X2; X2 = X3; X3 = newX0;
27    }
28
29    {
30        uint32_t arr[4] = {X0, X1, X2, X3};
31        sm4StoreBlock(arr, out);
32    }
33 }

```

编译运行结果截图如下:

```

orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orangeip@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_xbox
[PASS] sm4_xbox: test vector correct
SM4 Xbox Table Lookup - 1048576 bytes x 10 repeats
Elapsed : 365158 μs
Throughput: 27.385 MB/s

```

Figure 7: SM4 默认 Xbox

效率较 SM4 默认循环和循环展开都有所提升, 不过这种提前计算会增大程序的空间复杂度。

4.5.2 Xbox 合并

观察 (1) 式可以发现最后循环移位的位数是固定的 (分别为 24、16、8、0), 可以分别预计算这里的固定位数循环移位, 将原本运行时计算 ROL 改为编译时计算且运行时查表, 实现 Xbox 合并。即计算:

$$\begin{cases} XBox0[i] = L(S(i)) \\ XBox1[i] = ROL(L(S(i)), 8) \\ XBox2[i] = ROL(L(S(i)), 16) \\ XBox3[i] = ROL(L(S(i)), 24) \end{cases}$$

那么实际运行时各个位置查对应的表即可:

$$\begin{cases} ROL(Xbox[b_0], 24) = XBox3[b_0] \\ ROL(Xbox[b_1], 16) = XBox2[b_1] \\ ROL(Xbox[b_2], 8) = XBox1[b_2] \end{cases}$$

关键代码如下:

```

1 static uint32_t XBOX0[256];
2 static uint32_t XBOX1[256];
3 static uint32_t XBOX2[256];
4 static uint32_t XBOX3[256];
5 static int xbox_merged_initialized = 0;
6
7 static inline uint32_t ROL(uint32_t x, int n)
8 {
9     return (x << n) | (x >> (32 - n));
10 }
11
12 static void initXboxMerged(void)
13 {
14     if (xbox_merged_initialized) return;
15
16     for (int i = 0; i < 256; i++) {
17         XBOX0[i] = sm4L((uint32_t)sm4Sbox((uint8_t)i));
18         XBOX1[i] = ROL(XBOX0[i], 8);

```

```

19     XBOX2[i] = ROL(XBOX0[i], 16);
20     XBOX3[i] = ROL(XBOX0[i], 24);
21 }
22 xbox_merged_initialized = 1;
23 }
24
25 static void sm4EncryptBlock(const uint32_t rk[32],
26                             const uint8_t in[16],
27                             uint8_t out[16])
28 {
29     uint32_t X0, X1, X2, X3;
30     {
31         uint32_t arr[4];
32         sm4LoadBlock(in, arr);
33         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
34     }
35
36     initXboxMerged();
37
38     for (int i = 0; i < 32; i++) {
39         uint32_t t = X1 ^ X2 ^ X3 ^ rk[i];
40         uint32_t newX0 = X0 ^ XBOX3[(t >> 24) & 0xFF]
41                             ^ XBOX2[(t >> 16) & 0xFF]
42                             ^ XBOX1[(t >> 8) & 0xFF]
43                             ^ XBOX0[t & 0xFF];
44
45         X0 = X1; X1 = X2; X2 = X3; X3 = newX0;
46     }
47
48     {
49         uint32_t arr[4] = {X0, X1, X2, X3};
50         sm4StoreBlock(arr, out);
51     }
52 }

```

编译运行后结果如下:

```

orange@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orange@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_xbo
x_merged
[PASS] sm4_xbox_merged: test vector correct
SM4 Xbox Merged (4 tables) - 1048576 bytes x 10 repeats
Elapsed : 284581 μs
Throughput: 35.139 MB/s
orange@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ |

```

Figure 8: SM4XBox 合并

可以看到程序运行效率和吞吐量相较之前版本优化得到很大提升,但因为需要预计

算和存储 4 张表, 因此有总计 $4 * 256 * 4B = 4KB$ 的额外空间开销。

4.6 SM4 默认轮函数连续异或与减少指令依赖

在默认循环实现版本中每轮循环都有 $tmp = X1 \oplus X2 \oplus X3 \oplus rk[i]$, 由于异或操作满足结合律:

$$a \oplus b \oplus c \oplus d = (a \oplus b) \oplus (c \oplus d)$$

因此这里可以分别计算前两个变量的异或结果和后两个变量的异或结果, 分别存到临时变量中, 最后将这两个临时变量再求异或, 这样能减少指令之间的依赖性, 即可以同时求两个临时变量。否则, 连续异或时想要求后一个异或必须要等前一个异或运算结束, 指令依赖关系导致只能串行执行。在 SM4 每轮最后的换位赋值时可能产生写寄存器又马上读的依赖, 因此可以用临时变量存放 $X_1 \oplus X_2$ 的值, 这样能确保整个过程只发生寄存器移动而无需内存访问。具体代码如下:

```

1 static void sm4EncryptBlock(const uint32_t rk[32],
2                             const uint8_t in[16],
3                             uint8_t out[16])
4 {
5     uint32_t X0, X1, X2, X3;
6     {
7         uint32_t arr[4];
8         sm4LoadBlock(in, arr);
9         X0 = arr[0]; X1 = arr[1]; X2 = arr[2]; X3 = arr[3];
10    }
11
12    for (int i = 0; i < 32; i++) {
13        /* Break the XOR chain: X1^X2 and X3^rk are independent */
14        uint32_t tmp1 = X1 ^ X2;
15        uint32_t tmp2 = X3 ^ rk[i];
16        uint32_t tmp = tmp1 ^ tmp2;
17        uint32_t t = sm4T(tmp);
18        uint32_t newX0 = X0 ^ t;
19
20        /* Feistel rotation with temporaries */
21        uint32_t prevX1 = X1;
22        uint32_t prevX2 = X2;
23        X0 = prevX1;
24        X1 = prevX2;
25        X2 = X3;
26        X3 = newX0;
27    }
28
29    {
30        uint32_t arr[4] = {X0, X1, X2, X3};
31        sm4StoreBlock(arr, out);
32    }
33 }

```

编译运行结果如图:

```

orangeipi@orangeiprv2: ~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ls
sm4_default_loop sm4_reduced_dep sm4_unrolled sm4_xbox sm4_xbox_merged sm4_xbox_zbb sm4_zbb
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ ./sm4_reduced_dep
[PASS] sm4_reduced_dep: test vector correct
SM4 Reduced Dependency - 1048576 bytes x 10 repeats
Elapsed : 444762 µs
Throughput: 22.484 MB/s
orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ |

```

Figure 9: SM4 减少指令依赖

可以看到程序运行效率和吞吐量较默认循环版几乎相同, 优化非常有限。

4.7 默认字节序转换与 RISC-V B 拓展 rev8 转换

查看开发板 CPU 所支持的指令集如下图:

```

orangeipi@orangeiprv2:~/Principles-of-Computer-Systems-Experiment/work/openhitls/testcode/demo/build$ cat /proc/cpuinfo
processor       : 0
hart           : 0
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch         : ky, x60
mvendorid     : 0x710
marchid       : 0x8000000058000001
mimpid        : 0x1000000049772200

processor       : 1
hart           : 1
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch         : ky, x60
mvendorid     : 0x710
marchid       : 0x8000000058000001
mimpid        : 0x1000000049772200

processor       : 2
hart           : 2
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch         : ky, x60
mvendorid     : 0x710
marchid       : 0x8000000058000001
mimpid        : 0x1000000049772200

processor       : 3
hart           : 3
model name     : Ky(R) X1
isa            : rv64imafdcv_zicbom_zicboz_zicntr_zicnd_zicsr_zifencei_zihintpause_zihpm_zfh_zfhmin_zca_zcd_zba_zbb_zbc_zbs_zkt_zve32f_zve32x_
zve64d_zve64f_zve64x_zvfh_zvfhmin_zvkt_zscofpmf_zstc_zvinval_zvnapot_zvpbmt
mmu            : sv39
uarch         : ky, x60

```

Figure 10: 开发板 CPU 信息

可以看到该板支持 zbb, 因此可以在编译时启用该拓展。该拓展的优化原理在于:

- 对字节序转换的优化: 标准 SM4 中对串 01234567 的解释为:

01 23 45 67

即大端序, 而在 RISC-V 中却是小端序, 因此 sm4LoadBlock() 函数需要转换。普通实现时的代码

```
1 X =
```

```
2 (in[0]<<24)
3 |
4 (in[1]<<16)
5 |
6 (in[2]<<8)
7 |
8 in[3];
```

对应的汇编代码为:

```
1 lbu
2 slli
3
4 lbu
5 slli
6
7 lbu
8 slli
9
10 or
11 or
12 or
```